

Strategy Game-Playing with Size-Constrained State Abstraction

ABSTRACT

Playing strategy games is a challenging problem for artificial intelligence (AI). One of the major challenges is the large search space due to a diverse set of game components. In recent works, state abstraction has been applied to strategy game AI and has brought significant performance improvements. State abstraction techniques rely on reducing the search space, e.g., by aggregating similar states. However, the application of these abstractions is hindered because the quality of an abstraction is difficult to evaluate. Previous works hence abandon the abstraction in the middle of the search to not bias the search to a local optimum. Thus, a hyperparameter is introduced to decide the time to abandon the current state abstraction. In this work, we propose a size-constrained state abstraction (SCSA), an approach that limits the maximum number of nodes being grouped together. We found that with SCSA, the abstraction is not required to be abandoned. Our empirical results on three strategy games show that the SCSA agent outperforms the previous methods and yields robust performance over different games.

KEYWORDS

Game artificial intelligence, state abstraction, monte carlo tree search, planning

ACM Reference Format:

. 2022. Strategy Game-Playing with Size-Constrained State Abstraction. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (FDG '23)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Strategy games have helped advance the development of Artificial Intelligence (AI) to achieve significant progress in competing with human players [6, 28], AI-AI cooperation [8, 13, 30] and human-AI cooperation [5, 12, 13, 25]. However, these agents depend on deep reinforcement learning, where the neural networks are trained and tuned for a specific game. In another word, these agents have task-specific network parameters and hyperparameters. To apply these agents to new games, the training process needs to be repeated. Therefore, it is expensive to apply these agents to different games. Alternatively, search-based algorithms such as Monte Carlo Tree Search (MCTS) have shown outstanding performance in general video game-playing [16, 21, 23]. One of the most challenging problems for search-based algorithms is to search through a combinatorial search space that is typically used in strategy games. Thus,

the resulting state space and action space grow exponentially with the number of units under the player's control [19, 20]. In this work, we focus on the problem of the large state space and how it can be handled by search-based algorithms.

In strategy games such as Starcraft, a number of units (e.g. buildings, armies) are distributed on the map. The state space of these games is defined as the combination of unit properties (e.g. positions, health points) the player can observe. This combinatorial space increases exponentially with the number of game components (including the unit number and unit properties etc.). On top of that, most strategy video games have complex unit categories and each unit has a diverse set of properties. Together, they produce large state and action spaces resulting in a much larger branching factor when compared to other games. With a large branching factor, MCTS finds it difficult to explore the tree deeply for accurate action-value approximation. Thus it leads to poor performance in playing these games.

State abstraction [11, 14] is a powerful technique that helps MCTS solve large-scale planning problems. State abstraction methods focus on simplifying the search space. This is often achieved by aggregating similar states to enable information sharing between these states. In MCTS, we can use state abstraction to aggregate tree nodes from the same layer into groups, thus the tree becomes narrower. This enables MCTS to spend more time exploring deeper layers of the tree, thus, finding better solutions. In strategy games, state abstraction [7, 10, 26, 29] has been applied to reduce the search space and gained significant performance improvements.

Previous works [1, 17] built up theoretical results that the performance loss with state abstraction is bounded. However, due to the practical state abstraction being approximated from samples, its quality remains unknown. A recent work [29] observes that the quality of abstraction generated with small samples could harm the planning performance. Therefore, they proposed an *early stop* mechanism to abandon the constructed state abstraction at an early stage. A parameter is defined to decide when to abandon the state abstraction. In their experiment results, the value of this parameter can influence the performance significantly. Therefore, there are two shortcomings of setting such a parameter. First, its range depends on the budget (the number of forward model calls). With more budget, its range grows linearly. Second, we observe that the optimal values of this parameter vary in different games. These shortcomings make its application to different games expensive.

In this paper, we proposed the size-constrained state abstraction (SCSA) method (See Figure 1) to overcome the above-mentioned problems. SCSA limits the maximal number of nodes in the same node group. It achieves good performance without using an *early stop* across different strategy games. However, SCSA introduces another parameter: the maximal node group size. Surprisingly, we find that using the same value of this parameter, SCSA gains good performance across different games. We evaluate the SCSA agent in three strategy games using a common value of the size limit. It outperforms all the baseline agents in two simple games and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FDG '23, April 11–14, 2023, Lisbon, Portugal

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-XXXX-X/18/06...\$15.00

<https://doi.org/XXXXXXX.XXXXXXX>

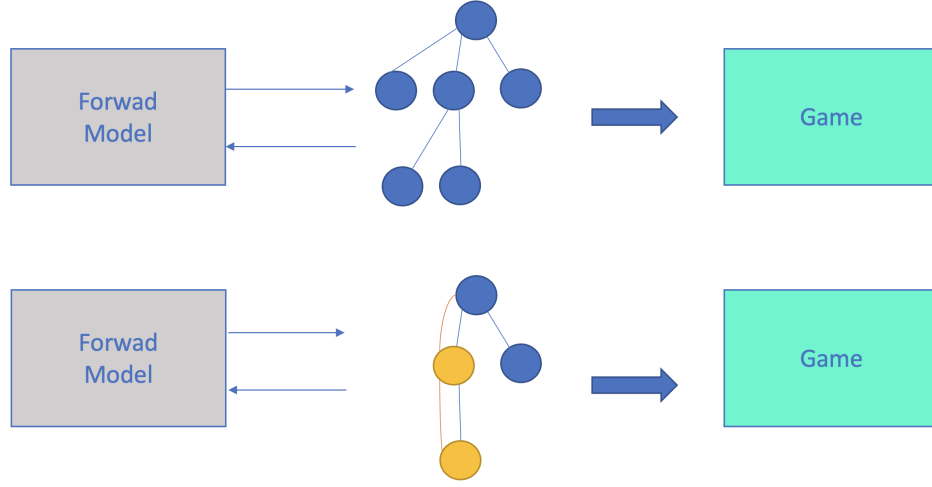


Figure 1: Monte Carlo Tree Search playing games with a forward model. The top figure shows the typical MCTS method. The bottom figure demonstrates the MCTS with the state abstraction that aggregates tree nodes into abstract nodes (yellow nodes). With state abstraction, the branching factor gets reduced.



Figure 2: A screenshot of *Kill The King* game. In this case, each player has one king, two warriors, two archers and two healers



Figure 3: A screenshot of *Push Them All* game. Each player has three pushers and try to push enemy units into holes.

achieves competitive results in another more complex strategy game.

The main contributions of this work are listed below:

- (1) We proposed a novel approach to tackle the unknown quality of the state abstraction in strategy game-playing.
- (2) We observed that the crucial parameter of the novel method is less sensitive to different applications. Our empirical results show that the same value of this parameter can achieve outstanding performance in different games.
- (3) We analyzed the compression rate under the SCSA and Elastic MCTS_u[29]. SCSA shows a lower compression rate, revealing the trade-off between memory usage and performance under the state abstraction.

2 RELATED WORK

Hostetler et al. [11] studies the optimal properties of state abstraction in MCTS. Jiang et al. [14] proposed to aggregate same-layer

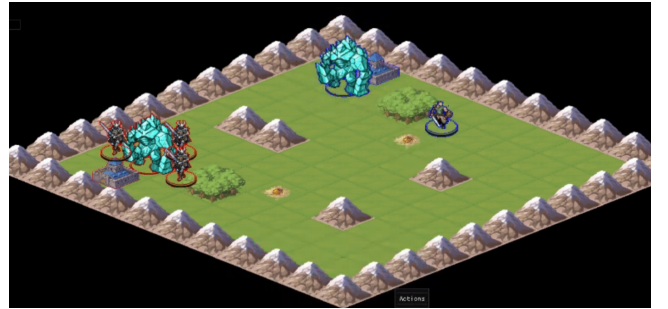


Figure 4: A screenshot of *Two Kingdom* game. The blue side spawned a worker to mine the gold and the red side spawned three warriors to protect the king.

tree nodes with MDP homomorphism approximated from samples. This abstraction expands with every batch of samples collected by the MCTS. This method shows a promising performance in the

board game Othello. Anand et al. [2] proposes a state-action abstraction method that aggregates state-action pair instead of states (tree nodes). Anand et al. [3] proposes progressive state abstraction that updates the state abstraction more frequently than using sample batches. Baier and Kaisers [4] proposes *abstraction over opponent moves* to aggregate tree nodes having the same opponent moving history. Sokota et al. [24] proposes *abstraction refinement* to reject similar states to be added in the tree. These methods prove that state abstraction is a powerful tool to tackle large branching factor in MCTS. However, their application is mainly in simple planning problems and board games.

State abstraction has already been applied to help strategy games-playing. Chung et al. [7] uses a handcrafted state abstraction to divide the game map into tiles. Synnaeve and Bessiere [26] proposed a mechanism to separate the map in StarCraft to regions that are connected through checkpoints. Uriarte and Ontanón [27] also uses the technique developed by Synnaeve and Bessiere [26] but further removed combat-irrelevant units from the map. These hand-crafted state abstraction can significantly reduce the size of state space thus obtained satisfactory search efficiency. Except for hand-crafted state abstraction, automatical state abstraction are also explored in strategy game-playing. Dockhorn et al. [10] proposed to represent game states with a combination of unit vectors. A parameter optimization method [18] is leveraged to search unit features that can be removed from the unit vectors. By removing some features, states having all other features the same are merged. Our work is closely related to Xu et al. [29], where an elastic MCTS method is proposed for strategy game-playing. In elastic MCTS, the state abstraction is first constructed with the approximate MDP homomorphism in a batch manner, similar to Jiang et al. [14]. Later, the constructed state abstraction is abandoned. Therefore abstract nodes are split into ground tree nodes and the remaining budgets are spent by the MCTS on ground nodes.

Using the *early stop* for abstraction usage is shown to obtain significant performance improvement. However, it involves a parameter whose range grows linearly with the given budgets. The SCSA method we proposed differs from Xu et al. [29] in that SCSA does not require *early stop*. i.e. the constructed state abstraction is kept until the end of MCTS running, similar to [2, 3, 11, 14]. Instead, a size constraint limit the maximal size of the abstract nodes.

3 THE STRATEGA PLATFORM

Stratega [9] is a general strategy game platform for testing AI agents. To evaluate the general performance of the proposed method, we select three two-player turn-based strategy games from the Stratega platform. They are *Kill The King (KTK)*, *Push Them All (PTA)* and *Two Kingdoms (TK)*. We next introduce details of these games.

In the *KTK* 2, each player controls a set of units including a *king*. The goal of this game is to kill the opponent's *king*. We instantiate the army for each player as a *king*, a *warrior*, an *archer* and a *healer*. All units have the move action. Based on that, the *king* and the *warrior* can attack neighbour enemy units. The *archer* can attack enemy units in range. The *healer* can heal ally units. Following Xu et al. [29], each unit also has an *Do-nothing* action. The action space size for a 4-unit army is about 10^5 .

In the *PTA* game 3, a player controls units to push enemy units in different directions. To win this game, all the enemy units need to be pushed into holes distributed in the map. Each player has 3 *pusher* units. The action set for each *pusher* is [*Move*, *Push*, *Do-nothing*], resulting in an action space of $(4+1) \times 4 \times 4 = 80$. The first term indicates moving in 4 directions or not moving, the second term is selecting a neighbour unit (there are 4 neighbour grids) and the last term is pushing the enemy unit in 4 different directions. With 3 *pushers*, the final action space is $80^3 = 512,000$.

The *TK* game is more complex. It consists of technologies, resources, unit spawning and combats. In the beginning, each player has a *castle* and a *king*. The aim is the same as *KTK*, i.e. kill the opponent *king*. However, a set of units need to be spawned from the *castle* for different tasks. A technology *Mining* is required to be researched for spawning *worker*. *Worker* unit can collect gold from gold veins and *Warrior*, *Night*, *Wizard* and *Healer* can be spawned with the collected gold.

4 BACKGROUND

4.1 Monte Carlo Tree Search

The MCTS is a method to solve sequential decision-making problems with a forward model. Given a state and a valid action under this state, the forward model returns the next state. With a limited number of calling the forward model, the MCTS build up a tree to approximate the value for actions under the current state. In the tree of MCTS, each node represents a game state and each branch represents a valid action of its source node. In each node, two statistics are stored. A *visit count* shows how many times this node is reached and a *score* is the sum of all scores from the back-propagation stage. We next introduce the 4 stages for building up this tree: *selection*, *expansion*, *rollout* and *back-propagation*.

In the *selection stage*, a tree node is selected for the subsequent stages. The target tree node could be a leaf node (a node that has no children) or a node where not all its actions have been added to the tree as branches. The *selection* starts from the root node and keeps selecting a branch to the next layer until a target node is reached. For each layer, a node with the highest UCB value [15] is selected. If the target node is a terminal state, the score of this state is used for the back-propagation stage. In another case, the target node is a non-terminal state, an action that has not yet been explored will be executed based on this state to get a new state, which is added to the current tree. Based on this new state, a roll-out policy takes a sequence of actions until a pre-determined depth or a terminal state is reached. This is the roll-out stage. The output state from rollout is evaluated by a state evaluation function to obtain a score. This score is used by the back-propagation stage. In the back-propagation stage, the score from the target state is added to all states in the trajectory of selection. i.e. a node sequence from the root node to the target node.

Each MCTS iteration consists of these 4 or 3 stages (the roll-out stage is skipped if the selected node is a terminal state). After running out of budgets, a recommendation policy selects an action to execute in the game. A common recommendation policy is selecting the branch leading to a node with the highest visit count.

4.2 Monte Carlo Tree Search with Unit Ordering

In strategy games where many units are distributed on the map, the action space for one state is the combination of all unit actions. e.g. in *KTK*, the combinatorial action space reaches a magnitude of 10^5 . To reduce the action space, Xu et al. [29] proposes the MCTS with unit ordering ($MCTS_u$). In $MCTS_u$, the move ordering of units is randomly initialized and is fixed throughout the whole game. Each node controls only one unit and its children control the subsequent unit in the move order. With this setting, the tree becomes deeper but narrower. The $MCTS_u$ has shown a strong performance in the multi-unit strategy game *KTK* in previous empirical analysis.

4.3 State Abstraction and Approximate MDP Homomorphism

An Markov Decision Process (MDP) is defined as $\langle \mathbb{S}, \mathbb{A}, R, P, \gamma \rangle$, where the $\mathbb{S}$ is the state space, $\mathbb{A}$ the action space, $R : \mathbb{S} \times \mathbb{A} \mapsto \mathbb{R}$ the reward function, $P : \mathbb{S} \times \mathbb{A} \mapsto \mathbb{S}$ the transition function and $\gamma \in \mathbb{R}$ is a discount factor. A state abstraction for an MDP is $\langle \mathbb{S}_\phi, \mathbb{A}, R, \hat{P}, \gamma \rangle$, where the $\hat{\mathbb{S}}$ is the abstract state space. Each abstract state includes a set of states. The $\hat{P} : \mathbb{S}_\phi \times \mathbb{A} \mapsto \mathbb{S}_\phi$ defines a transition function based on abstract states.

A key step to construct state abstraction is defining a state mapping function $\phi : \mathbb{S} \mapsto \mathbb{S}_\phi$ that maps a ground state to an abstract state. The function ϕ can be implemented by defining similarity between states and aggregating similar states in the same abstract state. Approximate MDP homomorphism [22] is a typical state similarity measurement. For two states s_1 and s_2 , it is defined by the approximate error of reward function $\epsilon_R(s_1, s_2)$ and the approximate error of transition function $\epsilon_T(s_1, s_2)$:

$$\epsilon_R(s_1, s_2) = \max_{a \in \mathbb{A}} |R(s_1, a) - R(s_2, a)| \quad (1)$$

$$\epsilon_T(s_1, s_2) = \max_{a \in \mathbb{A}} \sum_{s'_\phi \in \mathbb{S}_\phi} | \sum_{s' \in s'_\phi} T(s'|s_1, a) - \sum_{s' \in s'_\phi} T(s'|s_2, a) |, \quad (2)$$

where $\epsilon_R(s_1, s_2)$ measures the maximal difference between reward functions of the given states and $\epsilon_T(s_1, s_2)$ measures the worst-case total variation distance between transition distributions of the given states.

4.4 Elastic Monte Carlo Tree Search

The elastic MCTS method is built upon the $MCTS_u$. It aggregates tree nodes with approximate MDP homomorphism. The constructed node groups are split into ground tree nodes with *early stop*. Algorithm 1 and Algorithm 2 (without the blue parts) provide pseudo codes of elastic MCTS.

For every B MCTS iterations (line 6 in Algorithm 1), elastic MCTS checks all the tree nodes that have not yet been added in an abstraction node and calculate their approximate MDP homomorphism errors (line 9-10 in Algorithm 2). If the errors between the candidate state s_1 and an abstraction node $\hat{s}$ are below the pre-determined error thresholds η_R and η_T , s_1 is added into $\hat{s}$ (line 16). If there is no abstract node that matches this condition, a new abstract node is created with the s_1 as the only member node (line 18). The *early stop* shows in line 4-5 from Algorithm 1. It split all abstract nodes into ground nodes once the MCTS iteration reaches an *early stop* threshold α_{ES} .

5 METHOD

Based on MCTS, our method automatically groups tree nodes by the approximate MDP homomorphism. Following Jiang et al. [14] and Xu et al. [29], SCSA groups tree nodes from the same layer at every batch (a fixed number of MCTS iterations). At each iteration, the MCTS samples one trajectory that consists of a sequence of nodes, starting from the root node to a leaf node. To approximate the MDP homomorphism, a batch of samples is required for calculating the approximate errors (Formula 1). Therefore, for every B iterations, SCSA checks every tree node that has not yet joined a node group to expand the current abstraction. There are two approaches for a node to be added to the existing abstraction, depending on the value of approximate MDP homomorphism errors. If the approximate errors between this candidate tree node and a node group are below the thresholds, this candidate node is added to the corresponding node group, becoming a *member node* of this group. When the approximate errors between the candidate node and all same-layer groups are found higher than the thresholds, a new node group is created and this node becomes the only member node. As the state abstraction has both the approximate errors [14] for approximating MDP homomorphism and for approximating the transition function and reward function, a large number of samples are required to keep low approximate errors. In strategy games where the state space and action space are large, the collected samples typically achieve an unsatisfactory approximate accuracy. Moreover, it is difficult to evaluate the quality of the constructed state abstraction. Xu et al. [29] discovered that abandoning the existing state abstraction in the middle of MCTS running can bring significant performance improvement. Different from Xu et al. [29] where the constructed node groups are split into tree nodes after some iterations, SCSA does not split node groups. Instead, a global size constraint is defined to limit the maximum number of member nodes for every node group. Below, we further introduce the details of constructing and expanding the state abstraction during MCTS.

The pseudo codes of the SCSA algorithm are shown in Algorithm 1 and Algorithm 2, with highlighted lines in red are the reduced part from the elastic MCTS, and the highlighted lines in blue are added codes for the SCSA method. For convenience, we describe a node group as an abstract node. As the Algorithm 2 shows, an empty state abstraction ϕ , approximate error thresholds for the MDP homomorphism η_R and η_T and a size limit $SIZE_LIMIT$ are required. This function is called for every batch. States from the same layer are checked, from the bottom layers to the top layers (line 1-2). For a candidate node s_1 , its approximate errors with each existing same-layer abstract node are calculated (line 5-10). If the approximate errors between this candidate and an abstract node are found below the thresholds, at the same time the size of this abstraction node is within the limit defined by $SIZE_LIMIT$, the candidate node will be added into this abstract node as a member node (line 4-6 and line 11-13). If such a condition is not found after checking all same-layer abstract states, a new abstract node is created, where the candidate node is the only member node. Next, we introduce how this constructed state abstraction influences the MCTS.

In MCTS, each tree node stores the statistics of its value and visit count. These statistics are used to calculate the UCB score in

Algorithm 1 Elastic MCTS. $N_{fm} \in \mathbb{N}$: maximum forward model calls. $\alpha_{ABS} \in \mathbb{N}$: MCTS iteration threshold. $N_{mcts} \in \mathbb{N}$: current MCTS iteration number. $B \in \mathbb{N}$: batch size. $\phi(s) = \hat{s}$: state abstraction that maps an MCTS node representing s to a node group $\hat{s}$. $\eta_R \in \mathbb{R}$ and $\eta_T \in \mathbb{R}$ are the reward function error threshold and transition error threshold, respectively. *SIZE_LIMIT* is the maximal size of an abstract node.

Require: $N_{fm}, \alpha_{ABS}, \eta_R, \eta_T$

```

1:  $\phi := s \rightarrow \hat{s}, \hat{s} = \{s\}$  # Initialize the abstraction
2: while  $USED\_FMCALL < N_{fm}$  do
3:    $MCTSIteration(\phi)$ 
4:   if  $N_{mcts} > \alpha_{ES}$  then
5:      $\phi := s \rightarrow \hat{s}, \hat{s} = \{s\}$ 
6:   else if  $N_{mcts} \% B == 0$  then
7:      $\phi = ConstructAbstraction(\phi, \eta_R, \eta_T, SIZE\_LIMIT)$ 
8:   end if
9:    $N_{mcts} = N_{mcts} + 1$ 
10: end while

```

the selection stage and are updated in the back-propagation stage. When the recommender policy selects one action to execute in the game, it selects the first-layer node that has the maximum visit count. With a state abstraction, some nodes are grouped. Statistics for the abstract node are maintained. Once an abstract node is created, the value and the visit count of the only member node are assigned to this abstract node. Later on, the statistics of each new member node will add to the statistics of this abstract node. Each time the abstract node statistics update, increments are added to each member's statistics. Since the new member nodes will not be assigned by the abstract node statistics when it joins this abstract node, each member node has different statistics. These abstract node statistics influence the back-propagation stage, where the statistics of a sequence of nodes are updated. With the state abstraction, the abstract node statistics are also updated along with the member node from this sequence. All other member nodes are also updated because the abstract node statistics are updated. The abstraction changes the statistics update from one node to a group of nodes at each layer. These node statistics are used by the selection stage and action recommendation.

Note that SCSA introduces a new parameter *SIZE_LIMIT*. In practical application, we found a value of 2 is able to reach good performances in different games. Next, we introduce the experiments for performance evaluation, the influence of *SIZE_LIMIT* value and the tree size under SCSA.

6 EXPERIMENTS

Baselines: We implement 5 baseline agents to evaluate the performance of the SCSA agent. They are *Rule-based*, *MCTS*, *MCTS_u*, *RG MCTS_u* and *Elastic MCTS_u*. The details about each agent are listed below:

- (1) *Rule-based*: Stratega platform has implemented a Rule-based agent for each game. We here briefly introduce their implementation. Rule-based agent for *KTK* prioritize attacking isolated enemy units and healing strong ally units. For each enemy unit, an isolation score is calculated considering its

Algorithm 2 ConstructAbstraction($\phi, \eta_R, \eta_T, SIZE_LIMIT$), l is the tree depth and L is the maximum depth of the current tree.

```

1: for  $l = L$  to 1 do
2:   for all node  $s_1$  in depth  $l$  that is not grouped do
3:     for all abstract node  $\hat{s}$  in  $\phi$  do
4:       if  $|\hat{s}| > SIZE\_LIMIT$  then
5:         break
6:       end if
7:        $s_{1\_in\_}\hat{s} = \text{true}$ 
8:       for all node  $s_2$  in  $\hat{s}$  do
9:          $\epsilon_R = \max_a |R(s_1, a) - R(s_2, a)|$ 
10:         $\epsilon_T = \sum_{s'} |T(s'|s_1, a) - T(s'|s_2, a)|$ 
11:        if  $\epsilon_R > \eta_R$  or  $\epsilon_T > \eta_T$  then
12:           $s_{1\_in\_}\hat{s} = \text{false}$ , break
13:        end if
14:      end for
15:      if  $s_{1\_in\_}\hat{s} == \text{true}$  then
16:        Add  $s_1$  in abstract node  $\hat{s}$ 
17:      else
18:        Create a new abstract node
19:      end if
20:    end for
21:  end for
22: end for

```

nearby ally units and enemy units. At each round, Rule-based agent controls its units to i) approach the enemy units with the highest isolation score and attack them ii) approach an ally unit to heal it. The *PTA* Rule-based agent controls *pushers* to approach the nearest enemy unit and push it towards the nearest hole. In *TK*, the Rule-based agent first researches *Mining*, which is necessary for spawning *workers*. Once the research is finished, a *worker* is spawned and is assigned the task of collecting gold from the nearest gold vein. These gold are used to spawn *warriors*. Once the number of *warriors* reaches 2, these *warriors* are sent to attack the enemy *king*. Whenever its *warriors* died, new *warriors* will be spawned if the gold is enough.

- (2) *MCTS*: An MCTS agent without using state abstraction.
- (3) *MCTS_u*: An MCTS agent with unit ordering.
- (4) *RG MCTS_u*: An MCTS_u agent with randomized state abstraction. For each new tree node, it is uniformly added to one existing node group or creates a new node group with itself as the only *member node*.
- (5) *Elastic MCTS_u*: Elastic MCTS agent, an MCTS_u with state abstraction based on the approximate MDP homomorphism and *early stop*.
- (6) *SCSA*: An MCTS_u agent with approximate MDP homomorphism abstraction. Each abstract node has a size limit defined by *SIZE_LIMIT*.

Heuristic functions: The same as typical MCTS in games, we utilize heuristic functions to evaluate states reached by the MCTS roll-out. The heuristic functions are game-specific. In each game,

Table 1: Agent parameters for Section 6.2 experiment

Agents	C	L	α_{ABS}	η_R	η_T	SIZE_LIMIT
Kill The King (KTK)						
MCTS	0.1	10	/	/	/	/
MCTS _u	1.0	10	/	/	/	/
RG MCTS _u	0.1	10	8	/	/	/
Elastic MCTS _u	0.1	10	10	0.05	1.0	/
SCSA	0.1	10	/	0.05	1.0	2
Push Them All (PTA)						
MCTS	10	10	/	/	/	/
MCTS _u	10	20	/	/	/	/
RG MCTS _u	0.1	10	4	/	/	/
Elastic MCTS _u	10	10	8	1.0	1.0	/
SCSA	10	10	/	1.0	1.0	2
Two Kingdoms (TK)						
MCTS	0.1	20	/	/	/	/
MCTS _u	1.0	20	/	/	/	/
RG MCTS _u	0.1	10	8	/	/	/
Elastic MCTS _u	1.0	20	6	0.05	1.0	/
SCSA	1.0	20	/	0.05	1.0	2

all agents except for the Rule-based agent share the same heuristic function. Below, we introduce the implementation details of heuristic functions for each game.

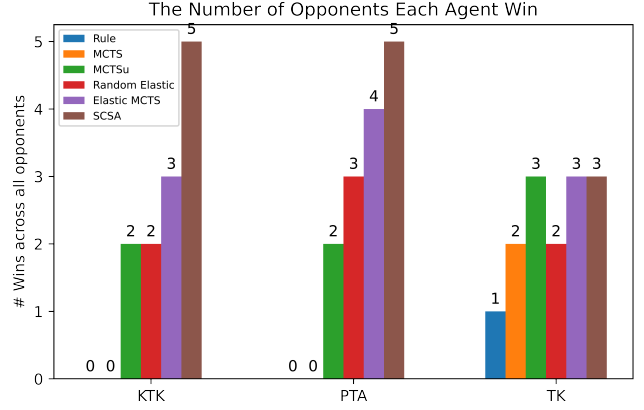
In all games, the score of states where the player wins, loses and draws the game give scores of 1, -1 and 0, respectively. For all the other states, the heuristic function returns a score between 0 and 1. The *KTK* heuristic function returns a score of $R = 1 - \frac{d \cdot h}{D \cdot H}$, where the d is the sum of the distance from each ally unit to the enemy king, D is the maximum value of d , h is the health points of the enemy king and D is the maximum value of d . The strategy is controlling the units to approach the enemy king and try to search for a state that leads to victory.

For *PTA*, the score of a state is separated into three parts. The first one is $0.2 \times \frac{\sum_u \min_{u'} dis(u, u')}{D}$, where the u is an ally unit, u' is an enemy unit, $dis(\cdot, \cdot)$ returns the Euclidean distance between the two units. The second part is $0.4 \times \frac{|U_t|}{|U_0|}$, where $|U_t|$ is the number of alive ally units at time step t and U_0 is the number of the units in the beginning. The last part is $0.4 \times \frac{|U'_0| - |U'_t|}{|U'_0|}$, where $|U'_t|$, $|U'_0|$ are the number of enemy units at time step t , respectively.

In *TK*, the state score is calculated according to the finishing of a series of tasks. Finishing *Mining* research returns 0.2, having *worker* alive returns 0.1 and having units that have action *attack* returns 0.1. Other scores include $0.1 \times$ the distance of ally workers to its nearest gold vein, $0.2 \times$ collected gold $0.3 \times$ the distance between all ally units and enemy units. The normalized score lands in $[0, 1]$.

6.1 Agent Parameter Optimisation with NTBEA

As each agent has different optimal parameters for each game, we apply the N-Tuple Bandit Evolutionary Algorithm (NTBEA) [18] to automatically optimize agent parameters in different games.

**Figure 5: The win numbers of different agents.**

The NTBEA use an N-Tuple system to break down the combinatorial space of the parameters. Therefore, it obtains satisfactory efficiency for parameter optimization. It has its own parameters, an exploration factor, the number of neighbours and the number of iterations. These values are set to 2, 50 and 50, respectively. Next, we introduce the parameter space for each game-playing agent.

The parameters for MCTS and MCTS_u are the exploration factor $C \in \{0.1, 1, 10, 100\}$ and rollout length $L \in \{10, 20, 40\}$. RG MCTS_u has C, L and an *early stop* threshold $\alpha_{ES} \in \{4 \times B, 8, 10, 12\}$, where the $B = 20$ is a constant batch size. Elastic MCTS_u has C, L, α_{ES} and approximate errors for reward function and transition function $\eta_R \in \{0.0, 0.04, 0.1, 0.3, 0.5, 1.0\}$, $\eta_T \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$. For SCSA, we use the same parameters as Elastic MCTS_u. It does not require the early stop threshold but requires a *SIZE_LIMIT*, which is linearly searched and set to 2 in the first group experiment.

The hyper-parameters tuned by NTBEA is demonstrated in Table 1.

Except for parameters, the budgets for search-based agents vary in different games. We set this budget based on the competitive performance of MCTS_u agent playing against the corresponding Rule-based agent. In *KTK* and *PTA* the budget is set to 10,000 number of forward model calls. In *TK*, the budget is 5,000 forward model calls.

6.2 Performance on general game-playing

To evaluate the general performance of SCSA agents, we ran an experiment with agents playing against each other in a two-player manner. For each agent pair, we use 5 random seeds and 50 games with side switching for each seed. This results in 500 games. This experiment consists of three games: *KTK*, *PTA* and *TK*.

The general performance of each agent is shown in Figure 5. In *KTK* and *PTA* games, SCSA outperforms all its opponents. In the more complex *TK* game, SCSA shows a competitive performance to the Elastic MCTS.

The detailed win rates for each agent pair are shown in Tables 2, 3 and 4. In *KTK*, the SCSA agent outperforms all its opponent agents in a significant gap. The MCTS shows a weaker performance than

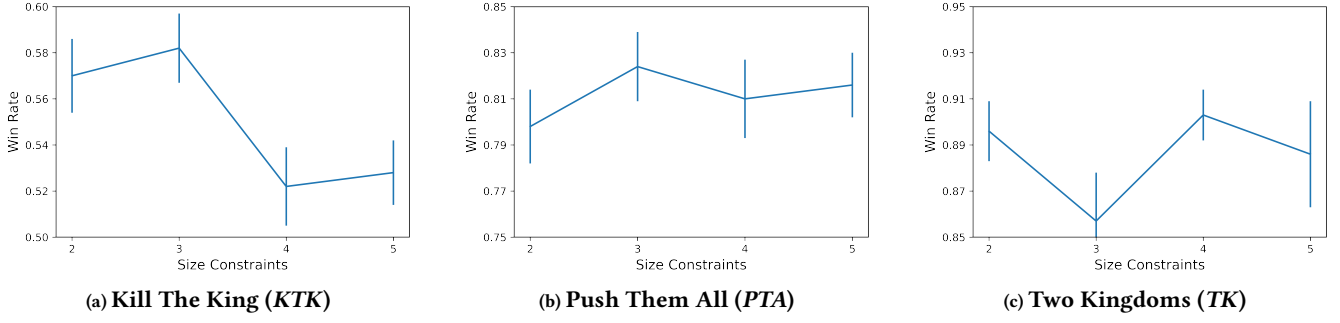


Figure 6: Performance of the SCSA agent with different values for its size constraint. Results of the SCSA agent playing against the corresponding Rule-based agents are visualized, including win rates and standard errors.

the rule-based agent. The $MCTS_u$ shows a stable and better performance than the MCTS. The Elastic $MCTS_u$ outperforms both the RG $MCTS_u$ and the $MCTS_u$.

In *PTA*, the overall win rates are higher than the *KTK*. In this game, SCSA agent also outperforms all its opponents. The Elastic $MCTS_u$ shows a strong performance in that it beats all agents except for the SCSA agent. The $MCTS_u$ outperforms the MCTS by a large margin. The MCTS agent is still weak than the rule-based agent.

In *TK*, the Rule-based agent outperforms the MCTS significantly while other agents outperform Rule-based with large margins. In this complex game, the $MCTS_u$, Elastic $MCTS_u$ and SCSA agents are showing close performances.

In conclusion, these experiments verify the performance improvement brought by the unit ordering and the elastic MCTS. It also evaluates the performance of the SCSA agent, confirming its outstanding performance in all three games. In addition, it shows good performance of SCSA agents in different games can be achieved by the same value for $SIZE_LIMIT$. In this experiment, we show that 2 is an appropriate value for $SIZE_LIMIT$. Comparing different games, the SCSA agent performs less strongly in the more complex *TK* game, indicating a potential issue of scalability.

6.3 Influence of abstract state size

To better investigate the influence of different values for $SIZE_LIMIT$, we assigned different values from 2 to 5 and run the agent pair of SCSA - Rule-based agent. The same as in Section 6.2, 500 games are run for each value of $SIZE_LIMIT$. Figure 6a-6c shows the win rates with standard errors in three games. We observe the optimal $SIZE_LIMIT$ values for *KTK* and *PTA* is 3 but *TK* has its optimal value at 2 and 4. We also observed that larger $SIZE_LIMIT$ values (e.g. a value of 4) can degrade the performance, which reveals the trade-off between the tree size and the performance. With a larger $SIZE_LIMIT$, more groups are aggregated together therefore the tree size becomes smaller. However, a too-small tree might bring performance degrades.

We used a value of 2 for all games and the agents shows satisfactory performance. Compared to different α_{ES} values are required in each domain (See Table V in Xu et al. [29]), the $SIZE_LIMIT$ is less sensitive across domains.

Table 2: Win rates with standard errors for games *Kill The Kings*

Agent 1	Agent 2	Agent 1	Agent 2
1 King, 1 Archer, 1 Warrior and 1 Healer			
MCTS	Rule-based	47.2(1.9)%	52.8(1.9)%
$MCTS_u$	Rule-based	62.2(1.1)%	37.6(1.3)%
RG $MCTS_u$	Rule-based	63.4(1.0)%	36.6(1.0)%
Elastic $MCTS_u$	Rule-based	54.2(1.4)%	44.8(1.6)%
SCSA	Rule-based	55.6(0.6)%	44.4(0.6)%
$MCTS_u$	MCTS	58.0(1.2)%	41.4(1.2)%
RG $MCTS_u$	MCTS	61.6(0.9)%	38.4(0.9)%
Elastic $MCTS_u$	MCTS	61.4(1.2)%	36.6(1.2)%
SCSA	MCTS	58.2(1.9)%	30.2(1.7)%
RG $MCTS_u$	$MCTS_u$	49.0(0.9)%	51.0(0.9)%
Elastic $MCTS_u$	$MCTS_u$	61.2(1.4)%	38.8(1.4)%
SCSA	$MCTS_u$	53.8(1.7)%	38.8(1.7)%
Elastic $MCTS_u$	RG $MCTS_u$	50.2(1.0)%	49.8(1.0)%
SCSA	RG $MCTS_u$	52.4(1.4)%	47.4(1.3)%
SCSA	Elastic $MCTS_u$	49.2(2.1)%	38.8(1.9)%

6.4 Compression Rate

To compare the influence of different state abstractions on tree size, we visualize compression rate at different MCTS iterations (See Figure 7a-7c). The compression rate is defined as the number of tree nodes dividing the number of abstract nodes. We can see that the $SIZE_LIMIT$ has constrained the compression rate by limiting the maximal size of each abstract node. Instead, the Elastic $MCTS_u$ agent can automatically control the compression. The overall compression rates of Elastic $MCTS_u$ are higher than SCSA and differ in different games. The vertical lines in Figure 7a-7c indicate the iteration when the Elastic $MCTS_u$ split all the node groups. Therefore, after this iteration, the compression rate for Elastic $MCTS_u$ is a constant 1.

We observe from the plots that the Elastic $MCTS_u$ without early stop can obtain a higher compression rate but this degrades

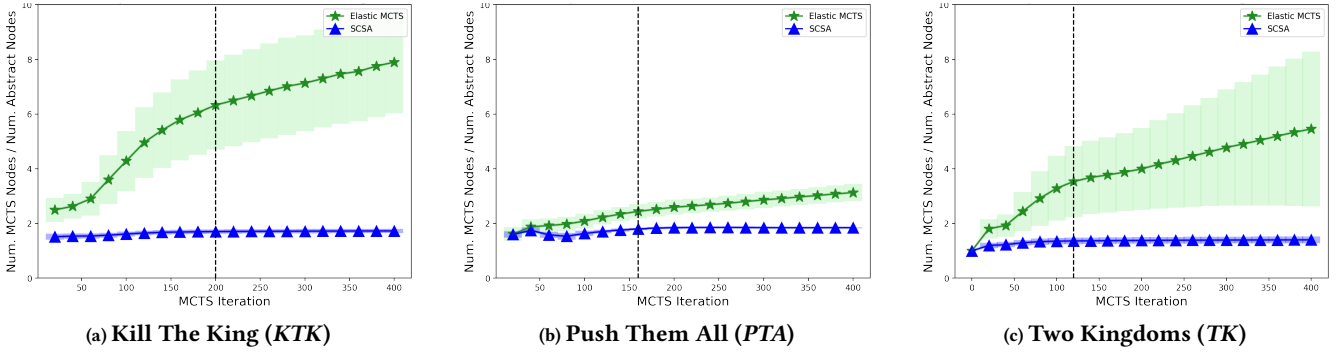


Figure 7: Compression rates for each tested game including standard errors from 10 game plays. The vertical line indicates the iteration when the abstraction is split into the original node for the Elastic MCTS_u.

Table 3: Win rates with standard deviation for games *Push Them All*

Agent 1	Agent 2	Agent 1	Agent 2
MCTS	Rule-based	48.8(1.6)%	51.2(1.6)%
MCTS _u	Rule-based	69.0(1.1)%	30.8(1.2)%
RG MCTS _u	Rule-based	74.0(2.3)%	26.0(2.3)%
Elastic MCTS _u	Rule-based	81.8(0.9)%	18.0(1.1)%
SCSA	Rule-based	79.8(1.6)%	20.2(1.6)%
MCTS _u	MCTS	64.4(1.1)%	33.6(1.4)%
RG MCTS _u	MCTS	86.2(0.7)%	12.0(0.7)%
Elastic MCTS _u	MCTS	85.4(1.5)%	13.0(1.7)%
SCSA	MCTS	86.0(1.4)%	12.8(0.9)%
RG MCTS _u	MCTS _u	73.4(2.0)%	25.6(1.9)%
Elastic MCTS _u	MCTS _u	80.2(1.0)%	18.0(1.0)%
SCSA	MCTS _u	77.2(1.8)%	22.0(2.0)%
Elastic MCTS _u	RG MCTS _u	62.4(1.4)%	35.8(1.3)%
SCSA	RG MCTS _u	58.8(1.8)%	40.4(2.0)%
SCSA	Elastic MCTS _u	52.0(1.6)%	46.8(1.9)%

the performance. The SCSA agent outperforms the Elastic MCTS_u in two of the three games and it has lower compression rates. These observations reveal a trade-off between the abstracted tree size and the agent performance.

7 CONCLUSION AND FUTURE WORK

Automatical state abstraction for MCTS is recently applied to overcome large search spaces in strategy game-playing. However, the use of unknown-quality state abstraction degrades the performance of MCTS. The previous solution is an *early stop* for state abstraction usage. This solution requires a parameter whose range grows linearly with the given search budgets. It is also sensitive to different games. These properties make it expensive to apply the state abstraction in different games. We proposed a novel size-constrained state abstraction to avoid the *early stop*. Instead, the size of the

Table 4: Win rates with standard deviation for games *Two Kingdoms*

Agent 1	Agent 2	Agent 1	Agent 2
MCTS	Rule-based	12.6(0.5)%	81.2(1.2)%
MCTS _u	Rule-based	90.8(1.7)%	7.6(1.6)%
RG MCTS _u	Rule-based	88.0(1.1)%	11.8(1.2)%
Elastic MCTS _u	Rule-based	89.2(1.4)%	9.6(1.3)%
SCSA	Rule-based	88.8(1.0)%	9.8(1.3)%
MCTS _u	MCTS	96.0(0.4)%	4.0(0.4)%
RG MCTS _u	MCTS	89.6(1.2)%	10.4(1.2)%
Elastic MCTS _u	MCTS	96.0(0.5)%	4.0(0.5)%
SCSA	MCTS	94.0(0.6)%	6.0(0.6)%
RG MCTS _u	MCTS _u	43.6(1.6)%	56.2(1.6)%
Elastic MCTS _u	MCTS _u	52.2(2.1)%	47.6(1.9)%
SCSA	MCTS _u	49.0(1.8)%	50.8(1.8)%
Elastic MCTS _u	RG MCTS _u	62.4(1.0)%	37.6(1.0)%
SCSA	RG MCTS _u	57.0(1.6)%	43.0(1.6)%
SCSA	Elastic MCTS _u	51.4(2.7)%	48.6(2.7)%

abstract node is limited. Our empirical results show that the SCSA agent shows satisfactory results in three strategy games with the same value of this size limit of 2. We analyze the influence of different size limits on the agent performance and the influence of different abstraction methods on the tree size, revealing the trade-off between the abstract tree size and agent performance. The SCSA outperforms baselines in two games but not in the complex *TK* game, indicating a potential shortcoming of scalability. We plan to further investigate the scalability of the SCSA agent in our future work.

REFERENCES

- [1] David Abel. 2019. A theory of state abstraction for reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 9876–9877.
- [2] Ankit Anand, Aditya Grover, Mausam Mausam, and Parag Singla. 2015. ASAP-UCT: Abstraction of State-Action Pairs in UCT. In *Proceedings of the 24th International Conference on Artificial Intelligence (Buenos Aires, Argentina) (IJCAI'15)*. AAAI Press, 1509–1515.

- [3] Ankit Anand, Ritesh Noothigattu, Parag Singla, et al. 2016. OGA-UCT: On-the-go abstractions in UCT. In *Proceedings of the International Conference on Automated Planning and Scheduling*, Vol. 26.
- [4] Hendrik Baier and Michael Kaisers. 2020. Guiding multiplayer mcts by focusing on yourself. In *2020 IEEE Conference on Games (CoG)*. IEEE, 550–557.
- [5] Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu, Athul Paul Jacob, Mojtaba Komeili, Karthik Konath, Minae Kwon, Adam Lerer, Mike Lewis, Alexander H. Miller, Sasha Mitts, Adithya Renduchintala, Stephen Roller, Dirk Rowe, Weiyan Shi, Joe Spisak, Alexander Wei, David Wu, Hugh Zhang, and Markus Zijlstra. 2022. Human-level play in the game of <i>Diplomacy</i> by combining language models with strategic reasoning. *Science* 0, 0 (2022), eade9097. <https://doi.org/10.1126/science.ade9097> arXiv:<https://www.science.org/doi/pdf/10.1126/science.ade9097>
- [6] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębniak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. 2019. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680* (2019).
- [7] Michael Chung, Michael Buro, and Jonathan Schaeffer. 2005. Monte Carlo Planning in RTS Games.. In *CIG*. Citeseer.
- [8] Brandon Cui, Hengyuan Hu, Luis Pineda, and Jakob Foerster. 2021. K-level Reasoning for Zero-Shot Coordination in Hanabi. *Advances in Neural Information Processing Systems* 34 (2021), 8215–8228.
- [9] Alexander Dockhorn, Jorge Hurtado Grueso, Dominik Jeurissen, and Diego Perez Liebana. 2020. STRATEGA: A General Strategy Games Framework.. In *AIIDE Workshops*.
- [10] Alexander Dockhorn, Jorge Hurtado-Grueso, Dominik Jeurissen, Linjie Xu, and Diego Perez-Liebana. 2021. Game state and action abstracting monte carlo tree search for general strategy game-playing. In *2021 IEEE Conference on Games (CoG)*. IEEE, 1–8.
- [11] Jesse Hostetler, Alan Fern, and Tom Dietterich. 2014. State aggregation in Monte Carlo tree search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 28.
- [12] Hengyuan Hu, Adam Lerer, Brandon Cui, Luis Pineda, Noam Brown, and Jakob Foerster. 2021. Off-belief learning. In *International Conference on Machine Learning*. PMLR, 4369–4379.
- [13] Hengyuan Hu, Adam Lerer, Alex Peysakhovich, and Jakob Foerster. 2020. “Other-Play” for Zero-Shot Coordination. In *International Conference on Machine Learning*. PMLR, 4399–4410.
- [14] Nan Jiang, Satinder Singh, and Richard Lewis. 2014. Improving UCT planning via approximate homomorphisms. In *Proceedings of the 2014 international conference on Autonomous agents and multi-agent systems*. 1289–1296.
- [15] Levente Kocsis and Csaba Szepesvári. 2006. Bandit based monte-carlo planning. In *European conference on machine learning*. Springer, 282–293.
- [16] John Levine, Clare Bates Congdon, Marc Ebner, Graham Kendall, Simon M. Lucas, Risto Miikkilainen, Tom Schaul, and Tommy Thompson. 2013. General Video Game Playing. In *Artificial and Computational Intelligence in Games*, Simon M. Lucas, Michael Mateas, Mike Preuss, Pieter Spronck, and Julian Togelius (Eds.). Dagstuhl Follow-Ups, Vol. 6. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 77–83. <https://doi.org/10.4230/DFU.Vol6.12191.77>
- [17] Lihong Li, Thomas J Walsh, and Michael L Littman. 2006. Towards a Unified Theory of State Abstraction for MDPs.. In *AI&M*.
- [18] Simon M Lucas, Jialin Liu, and Diego Perez-Liebana. 2018. The n-tuple bandit evolutionary algorithm for game agent optimisation. In *2018 IEEE Congress on Evolutionary Computation (CEC)*. 1–9.
- [19] Abdessamed Ouessai, Mohammed Salem, and Antonio M Mora. 2020. Improving the performance of mcts-based μ rts agents through move pruning. In *2020 IEEE Conference on Games (CoG)*. IEEE, 708–715.
- [20] Abdessamed Ouessai, Mohammed Salem, and Antonio Miguel Mora. 2020. Parametric action pre-selection for MCTS in real-time strategy games.. In *CoSECivi*. 104–115.
- [21] Diego Perez-Liebana, Spyridon Samothrakis, Julian Togelius, Tom Schaul, and Simon M Lucas. 2016. General video game ai: Competition, challenges and opportunities. In *Thirtieth AAAI conference on artificial intelligence*.
- [22] Balaraman Ravindran and Andrew G Barto. 2004. Approximate homomorphisms: A framework for non-exact minimization in Markov decision processes. (2004).
- [23] Chiara F Sironi, Jialin Liu, Diego Perez-Liebana, Raluca D Gaina, Ivan Bravi, Simon M Lucas, and Mark HM Winands. 2018. Self-adaptive mcts for general video game playing. In *International Conference on the Applications of Evolutionary Computation*. Springer, 358–375.
- [24] Samuel Sokota, Caleb Y Ho, Zaheen Ahmad, and J Zico Kolter. 2021. Monte carlo tree search with iteratively refining state abstractions. *Advances in Neural Information Processing Systems* 34 (2021), 18698–18709.
- [25] DJ Strouse, Kevin McKee, Matt Botvinick, Edward Hughes, and Richard Everett. 2021. Collaborating with humans without human data. *Advances in Neural Information Processing Systems* 34 (2021), 14502–14515.
- [26] Gabriel Synnaeve and Pierre Bessière. 2012. A Bayesian tactician. In *Computer Games Workshop at ECAI 2012*. pp–114.
- [27] Alberto Uriarte and Santiago Ontanón. 2014. Game-tree search over high-level game states in RTS games. In *Tenth Artificial Intelligence and Interactive Digital Entertainment Conference*.
- [28] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* 575, 7782 (2019), 350–354.
- [29] Linjie Xu, Jorge Hurtado Grueso, Dominik Jeurissen, Diego Perez Liebana, and Alexander Dockhorn. 2022. Elastic Monte Carlo Tree Search with State Abstraction for Strategy Game Playing. In *IEEE Conference on Games, CoG 2022, Beijing, China, August 21-24, 2022*. IEEE, 369–376. <https://doi.org/10.1109/CoG51982.2022.9893587>
- [30] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. 2022. The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=YVXaxB6L2Pl>